

CAMPUS RESOURCE HUB: A CENTRALIZED WEB PLATFORM FOR ACADEMIC RESOURCE SHARING WITH AI-POWERED MODERATION

Akib Rayhan (ID. 42230200858)

Rakibul Hasan (ID. 42230200859)

Mahdi Hasan (ID. 42230200717)

A Project Report Submitted in Partial Fulfillment of the Requirements for the
Degree of Bachelor of Science in Computer Science & Engineering



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
NORTHERN UNIVERSITY BANGLADESH

October 2026

CAMPUS RESOURCE HUB: A CENTRALIZED WEB PLATFORM FOR ACADEMIC RESOURCE SHARING WITH AI-POWERED MODERATION

Akib Rayhan (ID. 42230200858)

Rakibul Hasan (ID. 42230200859)

Mahdi Hasan (ID. 42230200717)

A Project Report Submitted in Partial Fulfillment of the Requirements for the
Degree of Bachelor of Science in Computer Science & Engineering



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
NORTHERN UNIVERSITY BANGLADESH

October 2026

APPROVAL

The Project Report on “CAMPUS RESOURCE HUB: A CENTRALIZED WEB PLATFORM FOR ACADEMIC RESOURCE SHARING WITH AI-POWERED MODERATION” submitted by Akib Rayhan, ID: 42230200858, Rakibul Hasan, ID: 42230200859, and Mahdi Hasan, ID: 42230200717, to the Department of Computer Science and Engineering, Northern University Bangladesh, has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering and approved as to its style and contents.

Supervisor

Tanmoy Mazumder
Lecturer, Department of Computer Science and Engineering
Northern University Bangladesh

Course Coordinator

Rakibul Hasan
Lecturer, Department of Computer Science and Engineering
Northern University Bangladesh

Project Coordinator

Moinuddin
Lecturer, Department of Computer Science and Engineering
Northern University Bangladesh

Md. Raihan-ul-Masood
Associate Professor and Head, Department of Computer Science and Engineering
Northern University Bangladesh

DECLARATION

We hereby declare that the work presented in this project report is the outcome of the investigation performed by us under the supervision of Tanmoy Mazumder, Lecturer, Department of Computer Science and Engineering, Northern University Bangladesh. We also declare that no part of this project has been or is being submitted elsewhere for the award of any degree or diploma.

Akib Rayhan
Student No. 42230200858

Rakibul Hasan
Student No. 42230200859

Mahdi Hasan
Student No. 42230200717

Abstract

University students commonly rely on disconnected social groups, messaging applications, cloud folders, and notice boards to exchange academic materials and campus information. This fragmentation makes resources difficult to discover, weakens moderation, and separates information from actions such as joining a club, registering for an event, or claiming a lost item. This project aimed to design and implement Campus Resource Hub, a centralized platform that combines academic resource sharing with campus services, role-based administration, real-time communication, and artificial-intelligence-assisted retrieval. The system uses a React single-page web application, a Node.js and Express application programming interface, MongoDB for document and vector data, Cloudinary for managed file storage, and Socket.IO for real-time functions. Verified users can publish in-app text resources or upload supported files, while safety-sensitive content follows automated screening and human review paths. The platform also provides events with capacity-aware responses, club membership workflows, privacy-preserving lost-and-found claims, scheduled announcements, direct and group messaging, notifications, and a semantic assistant built on 384-dimensional multilingual sentence embeddings. Repository verification completed 27 backend tests, 6 frontend tests, and 1 mobile test successfully. The implemented system demonstrates that academic resources and routine campus workflows can be consolidated in a secure, searchable, and extensible application, while current limitations in third-party service quotas and production-scale load validation identify clear priorities for future deployment work.

Keywords: campus resource sharing, semantic search, content moderation, role-based access control, real-time communication, full-stack web application.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to everyone who supported the planning, development, testing, and documentation of this project.

First and foremost, we extend our deepest appreciation to our supervisor, Tanmoy Mazumder, Lecturer in the Department of Computer Science and Engineering at Northern University Bangladesh, for his guidance, constructive feedback, and encouragement throughout this work. His advice helped us improve both the technical implementation and the presentation of the project.

We are also grateful to the respected faculty members of the Department of Computer Science and Engineering for the knowledge and academic support that prepared us to undertake this project. We thank the department and Northern University Bangladesh for providing the environment and resources needed to complete our undergraduate work.

Our thanks extend to the developers and maintainers of the open-source frameworks, libraries, documentation, and cloud services used in Campus Resource Hub. We also appreciate the classmates and students whose practical observations helped us understand real campus communication and resource-sharing needs.

Finally, we acknowledge the patience, prayers, and moral support of our families and friends. We also thank one another as team members for sharing responsibilities, resolving technical challenges, and remaining committed through every stage of the project.

Contents

Approval	i
Declaration	ii
Abstract	iii
Acknowledgements	iv
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	2
1.4 Proposed Method	2
1.5 Organization of the Report	3
2 Related Previous Work	4
2.1 Discussion on Existing Solutions	4
2.2 Research Gap	5
2.3 Summary of Existing Systems	5
2.4 Research Positioning and Contribution	6
3 Background Study	7
3.1 Topic Overview	7
3.1.1 MERN-Style Web Architecture	7
3.1.2 REST API and JWT Authentication	7
3.1.3 WebSocket and Real-Time Communication	7

3.1.4	AI Content Moderation	7
3.1.5	Sentence Embeddings and Semantic Search	8
3.1.6	Cloud Object Storage	8
3.2	Technology Stack	8
4	System Requirements and Design	9
4.1	Functional Requirements	9
4.2	Non-Functional Requirements	9
4.3	System Design	10
4.3.1	System Architecture	10
4.3.2	System Workflow	10
4.3.3	User Roles	10
4.4	Database Design	12
4.5	API Design	12
4.6	Security Design	14
5	Methodology	15
5.1	Implementation Workflow	15
5.2	Authentication and OTP Verification	15
5.3	Resource Upload and AI Moderation Pipeline	16
5.4	Events with Capacity-Aware RSVP	16
5.5	Clubs with Join Policies and Member Roles	18
5.6	Lost and Found with Claim Workflow	19
5.7	Announcements with Scheduled Publishing	19
5.8	Real-Time Chat	19
5.9	Notifications	21
5.10	Semantic Search and AI Assistant	21
5.11	Admin Panel	21
5.12	Deployment	21
6	Results and Analysis	23

6.1	Achieved Features	23
6.2	Security Analysis	23
6.3	Bug Report and Resolution Log	24
6.3.1	Current Verification Snapshot	26
6.3.2	Open Operational Constraint	26
6.4	Verification and Capacity Limitations	26
6.5	Cost and Deployment Considerations	27
Conclusion and Future Work		28
References		29
A Supplementary Material		30
A.1	Code Availability	30
A.2	Current Automated Verification	30

List of Figures

4.1	Three-tier system architecture, data services, and external integrations	10
4.2	User access, resource moderation, publication, campus services, and notification workflow	11
4.3	MongoDB collections for authorization, academic content, communication, and AI-assisted retrieval	13
5.1	Role-aware dashboard with activity summary cards, recent resources, and latest announcements	15
5.2	Login and signup with e-mail OTP verification	16
5.3	Resource upload with metadata and moderation status	17
5.4	Browsing, filtering, previewing and downloading published resources	17
5.5	Event listing and RSVP flow	18
5.6	Club directory and membership management	18
5.7	Lost-and-found listing and claim dialog	19
5.8	Announcement feed with priority badges	20
5.9	Real-time direct and group messaging with seen receipts	20
5.10	Admin panel: statistics and review tools	22

List of Tables

2.1	Comparison of related solutions with Campus Resource Hub	6
3.1	Technology stack of Campus Resource Hub	8
4.1	Principal functional requirements	9
4.2	User roles and their permissions	11
4.3	Summary of database collections	12
4.4	REST API route groups (representative endpoints)	14
6.1	Feature completion summary	23
6.2	Resolved defect log derived from Git history and regression tests	24
6.3	Automated test-suite results for the current repositories	27

Chapter 1

Introduction

University students constantly exchange study materials, event information, club activities, and everyday campus updates. In most institutions this exchange is scattered across social media groups, messaging apps, email threads, and physical notice boards. Important files get buried in chat histories, event announcements never reach everyone, found items rarely return to their owners, and there is no quality control over what is shared. *Campus Resource Hub* (CRH) addresses this fragmentation by providing one moderated, searchable, real-time platform for the entire campus community.

1.1 Motivation

The motivation for this project came from observing how students in our own department share resources today: class notes travel through Facebook Messenger groups, seminar notices are photocopied onto notice boards, and lost ID cards are announced verbally. Each channel reaches only a fraction of students, content disappears quickly, and nothing is verified. A purpose-built platform can solve all of these problems at once, while also applying modern techniques — such as AI content moderation and semantic search — that general-purpose tools do not offer for campus use.

1.2 Problem Statement

The specific problems this project solves are:

1. Study materials have no central, categorized, searchable repository; files are duplicated and lost across chat groups.
2. Campus events lack a unified registration system with capacity control and attendance tracking.
3. Club membership management (joining, approval, member roles) is handled manually.
4. Lost-and-found has no structured claim-and-verify process, and posting personal contact information publicly is unsafe.
5. Announcements cannot be scheduled, prioritized, or tracked for readership.

6. Shared content is completely unmoderated; harmful or inappropriate uploads spread unchecked.
7. Keyword search cannot find content expressed in different words, and there is no way to *ask questions* about campus information.

1.3 Research Objectives

The objectives of the project are to:

- Build a centralized web platform where verified students can create structured text resources and upload, browse, preview, and download categorized study files.
- Implement secure authentication with email OTP verification, JWT sessions, and role-based access control (student, moderator, admin).
- Provide automated content-safety screening and human review for resources and other safety-sensitive publishing flows.
- Support campus events with capacity-aware RSVP, club management with join policies and member roles, and a privacy-preserving lost-and-found claim system.
- Deliver real-time direct and group messaging with typing indicators and seen receipts using WebSocket technology.
- Implement semantic (meaning-based) search over all published content using sentence embeddings and vector search, exposed through a streaming AI assistant.
- Deploy the complete system on cloud infrastructure at minimal cost.

1.4 Proposed Method

The project followed an iterative full-stack development method. Requirements were translated into independently testable modules, a React client was connected to an Express application programming interface, and MongoDB schemas and authorization middleware were implemented for each workflow. Uploaded and authored resources were processed through validation, safety checks, storage, and indexing stages. Functional behavior was evaluated with automated backend, frontend, and mobile test suites together with manual role-based workflow checks. The system architecture and requirements are described in Chapter 4, while the implementation process is detailed in Chapter 5.

1.5 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 reviews related systems and identifies the research gap. Chapter 3 explains the technical background. Chapter 4 presents the requirements, architecture, data model, application programming interface, and security design. Chapter 5 describes the implementation of the major modules. Chapter 6 reports verification results, resolved defects, and deployment limitations. The final chapter summarizes the work and outlines future improvements, followed by references and supplementary repository information.

Chapter 2

Related Previous Work

2.1 Discussion on Existing Solutions

Several categories of existing tools partially address campus information sharing. Each category solves a useful part of the problem, but none provides a complete student-centered campus hub.

Social media groups (Facebook Groups, Messenger, WhatsApp). Social platforms provide broad group communication and file sharing [1, 2]. These are the de-facto standard in most universities because they are free and familiar. However, they mix academic content with unrelated posts, provide no file categorization (a lecture note from last semester is effectively unfindable), have no moderation tailored to academic norms, and expose members’ personal profiles. Search is normally limited to message text, so students must remember who posted a file or approximately when it appeared. Ownership and moderation also depend on informal group administrators rather than institutional roles.

Learning Management Systems (Moodle, Google Classroom). These systems organize course activities and teacher-managed materials [3, 4]. LMS platforms organize course content well, but they are teacher-centric: students cannot freely share peer resources, and campus-life features — clubs, events with RSVP, lost-and-found, peer chat — are out of scope. Institutional LMS deployments also carry licensing and administration overhead. Their course-by-course structure is effective for formal teaching, yet it fragments cross-department resources and does not provide a unified discovery experience for the wider campus community.

Generic cloud drives (Google Drive, OneDrive). Cloud drives provide shared storage and link-based collaboration [5]. Shared folders solve storage but not discovery: there is no moderation, no metadata like course/semester, and links rot as owners graduate. Access is often controlled through manually shared links, which makes permission management inconsistent and gives students little confidence that a document is current, safe, or academically relevant.

University notice boards and websites. Official channels are one-directional, updated slowly, and offer no interactivity or personalization. They remain valuable for authoritative notices, but usually do not support student contributions, targeted department feeds, event registration, real-time updates, or searchable archives.

Specialized campus applications. Some institutions deploy separate systems for events, library services, student clubs, or lost-and-found reporting. Such applications provide deeper

support for one workflow, but students must maintain several accounts and move between disconnected interfaces. Data produced in one service cannot normally improve search or recommendations in another.

2.2 Research Gap

The review reveals five recurring gaps. First, information is **fragmented**: academic files, events, notices, and community activities live in separate channels. Second, discovery is mostly **keyword- or time-dependent**; users cannot ask a natural language question and retrieve relevant passages from inside files. Third, peer-contributed content often lacks a reliable **safety and approval process**. Fourth, the systems provide inconsistent **privacy and role controls**, especially for personal contact information and student-generated posts. Finally, many products omit the **interactive campus workflows** that connect information with action, such as registering for an event, joining a club, claiming a found item, or messaging another user.

These gaps are not independent. For example, adding cloud storage without metadata improves availability but not discovery; adding a search box without moderation may make unsafe material easier to find; and publishing notices without notifications does not ensure that the intended students see them. A useful campus platform must therefore treat storage, retrieval, safety, authorization, and interaction as parts of one architecture.

2.3 Summary of Existing Systems

Campus Resource Hub combines the useful parts of each category into one student-centric platform and adds capabilities none of them provide in this context:

- **Structured resource repository** with course, department, and semester metadata, view and download analytics, and in-browser preview.
- **Fail-safe two-layer moderation**: an automated AI safety scan at upload time, with unsafe, partial, or provider-unavailable checks held for moderator review.
- **Campus-life modules** (events with capacity-aware RSVP, clubs with join policies and officer roles, claim-based lost-and-found with private contact reveal) that LMS platforms do not cover.
- **Semantic search and an AI assistant** over all campus content, using sentence embeddings and vector search [6, 7, 8] — beyond basic keyword-only retrieval.
- **Zero-cost deployment** on free cloud tiers, making the system realistic for a public university budget.

Table 2.1 summarizes the functional positioning. “Yes” means that the capability is a normal part of the category; “Limited” means that it depends on manual setup, separate tools, or basic keyword matching.

Table 2.1: Comparison of related solutions with Campus Resource Hub

Capability	Social media	LMS	Cloud drive	Notice site	CRH
Structured academic metadata	No	Yes	Limited	No	Yes
Student resource contribution	Yes	Limited	Yes	No	Yes
Automated safety moderation	No	No	No	No	Yes
Semantic file-content search	No	Limited	Limited	No	Yes
Events, clubs, and lost-found	Limited	No	No	Limited	Yes
Role-aware privacy controls	Limited	Yes	Limited	No	Yes
Real-time chat and alerts	Yes	Limited	Limited	No	Yes
Numbered AI answer sources	No	No	No	No	Yes

2.4 Research Positioning and Contribution

The proposed work is not intended to replace an LMS or an official university website. Instead, it occupies the space between formal teaching systems and informal student communication. It preserves the low-friction contribution model of social platforms, the organization of an LMS, the storage flexibility of cloud drives, and the authority controls of institutional systems within one role-aware application.

Its principal contribution is the integration of these established ideas with two applied-AI workflows. The first inspects uploaded text and images before publication and fails safely to human review when automatic inspection is incomplete. The second indexes metadata and ordered passages from PDF, Office, and safe image resources, enabling grounded questions and summaries with visible sources. Combining these workflows with events, clubs, lost-and-found, notifications, and real-time messaging makes the system broader than a document repository while remaining focused on everyday university use.

This comparison also defines the evaluation criteria used later in the report: discoverability, publication safety, authorization, workflow completeness, responsiveness, and deployment cost. The Result and Analysis chapter evaluates the implemented platform against these criteria rather than claiming that any single existing product is universally inadequate.

Chapter 3

Background Study

3.1 Topic Overview

Campus Resource Hub combines conventional web application architecture with real-time communication, managed storage, automated content screening, and semantic information retrieval. This chapter introduces the concepts needed to understand the system design.

3.1.1 MERN-Style Web Architecture

The system follows the popular JavaScript full-stack pattern: **MongoDB** as the document database, **Express** as the HTTP framework, **React** as the frontend library, and **Node.js** as the server runtime. The frontend is a single-page application (SPA) that communicates with the backend exclusively through a REST API, keeping the two layers independently deployable [9, 10, 11].

3.1.2 REST API and JWT Authentication

The backend exposes resources (users, files, events, clubs, etc.) through RESTful endpoints. After login, the server issues a *JSON Web Token* (JWT) — a signed token the client attaches to every request. Middleware verifies the signature and the user’s role before allowing access, so protected operations never depend on server-side session storage [12].

3.1.3 WebSocket and Real-Time Communication

HTTP is request–response; the server cannot push data to a client. *WebSocket* keeps a persistent two-way connection open, and the *Socket.io* library adds rooms, acknowledgements, and automatic fallback. CRH uses it for instant message delivery, live seen/typing indicators, and real-time notifications [13].

3.1.4 AI Content Moderation

Every uploaded file’s text and images are extracted and sent to a large-language-model moderation service, which classifies content against safety categories (sexual content, violence, ha-

rassment, etc.). Flagged uploads are quarantined for human review instead of being published — a two-layer (AI + human) moderation pipeline.

3.1.5 Sentence Embeddings and Semantic Search

An *embedding* maps a piece of text to a numeric vector such that semantically similar texts land near each other in vector space. CRH uses a multilingual MiniLM sentence-transformer model to produce 384-dimensional vectors for every published item, stores them in a dedicated collection, and queries them with MongoDB Atlas Vector Search (k -nearest-neighbour cosine similarity). This lets a search for “exam preparation notes” find a document titled “final term study guide” — something keyword search cannot do [6, 7, 8].

3.1.6 Cloud Object Storage

User files are not stored on the application server; they are uploaded to Cloudinary, a cloud media service, and the database only keeps secure URLs and public IDs. This keeps the server stateless and lets the platform scale storage independently [14].

3.2 Technology Stack

The complete technology stack is listed in Table 3.1. All layers use JavaScript, which keeps the codebase uniform and allows shared conventions between frontend and backend.

Table 3.1: Technology stack of Campus Resource Hub

Layer	Technology	Purpose
Frontend	React 19.2 + Vite 7	Single-page application UI
	React Router	Client-side routing, protected routes
Backend	Node.js + Express 5	REST API server
	Socket.io	Real-time messaging and notifications
Database	MongoDB Atlas + Mongoose	Document storage, schemas, indexes
	Atlas Vector Search	384-dim semantic search index
AI	Groq / Hugging Face / OpenAI	Content-safety provider chain
	Transformers.js (MiniLM)	Local sentence-embedding generation
Storage	Cloudinary	File and image hosting
Auth	JWT + bcrypt	Stateless sessions, password hashing
E-mail	Gmail API / SendGrid / SMTP	OTP e-mail delivery with fallbacks
Hosting	Vercel frontend / backend	Managed deployment and serverless API

Chapter 4

System Requirements and Design

4.1 Functional Requirements

The functional requirements were derived from the recurring academic and campus workflows identified during project planning. Table 4.1 summarizes the required behavior that guided implementation and verification.

Table 4.1: Principal functional requirements

ID	Requirement
FR-01	Register verified users through e-mail one-time-password confirmation and authenticate them with role-aware access.
FR-02	Create text resources and upload supported files with academic metadata, preview, filtering, and download controls.
FR-03	Screen safety-sensitive content and route uncertain or flagged submissions to authorized human review.
FR-04	Support events, clubs, announcements, and privacy-preserving lost-and-found claims in one application.
FR-05	Provide direct and group messaging, typing indicators, seen state, and in-application notifications.
FR-06	Retrieve approved content through keyword and semantic search and present grounded assistant sources.
FR-07	Provide moderator and administrator tools for users, resources, lost-and-found items, and platform statistics.

4.2 Non-Functional Requirements

The application was designed for security, maintainability, usability, and graceful degradation. Authentication data must remain protected, authorization must be enforced on the server, unavailable artificial-intelligence providers must not cause unsafe auto-publication, and independent web, API, database, storage, and mobile layers must remain replaceable. Interfaces must be responsive and understandable across common desktop and mobile sizes, while logs, tests, and content hashes must support diagnosis and repeatable verification.

4.3 System Design

4.3.1 System Architecture

The platform uses a three-tier architecture (Figure 4.1). The presentation tier is a React SPA served through Vercel. The application tier is a Node.js/Express application deployed through Vercel-compatible serverless entry points that exposes the REST API and the Socket.io gateway. The data tier consists of MongoDB Atlas (documents and vector index), Cloudinary (file objects), and external service providers for email (Gmail API with SendGrid and SMTP fallbacks) and AI moderation (Groq, Hugging Face, and OpenAI fallbacks).

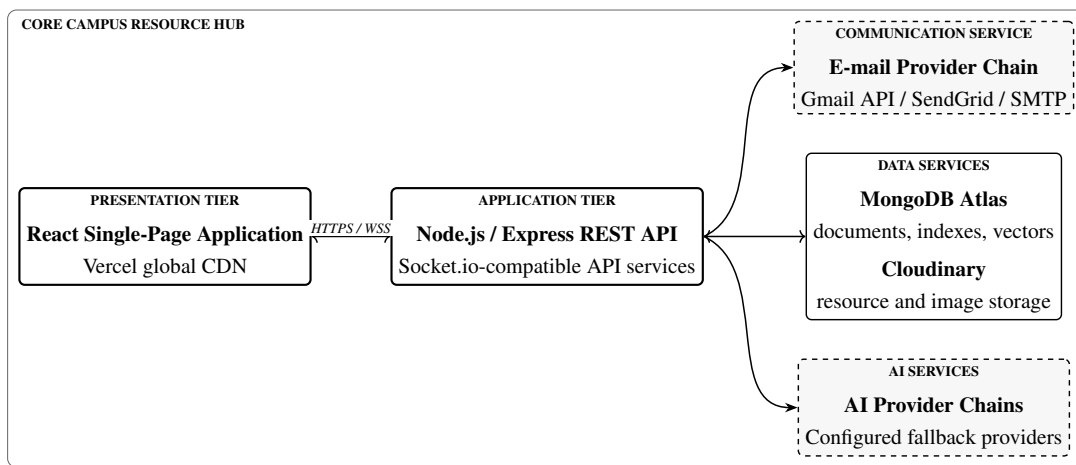


Figure 4.1: Three-tier system architecture, data services, and external integrations

4.3.2 System Workflow

Figure 4.2 shows the complete system workflow. After OTP-verified signup and JWT login, users reach a role-aware dashboard from which all eleven functional flows branch out: resources, events, clubs, lost-and-found, announcements, chat, notifications, AI search, admin panel, and profile management. The central design pattern shared by moderated content flows is *create* → *safety scan* → *auto-publish when fully clean*, otherwise *human review* → *notify*.

4.3.3 User Roles

Access control follows a three-tier role model. Table 4.2 summarizes what each role is allowed to do; every privileged API route enforces these rules through the `authorize` middleware.

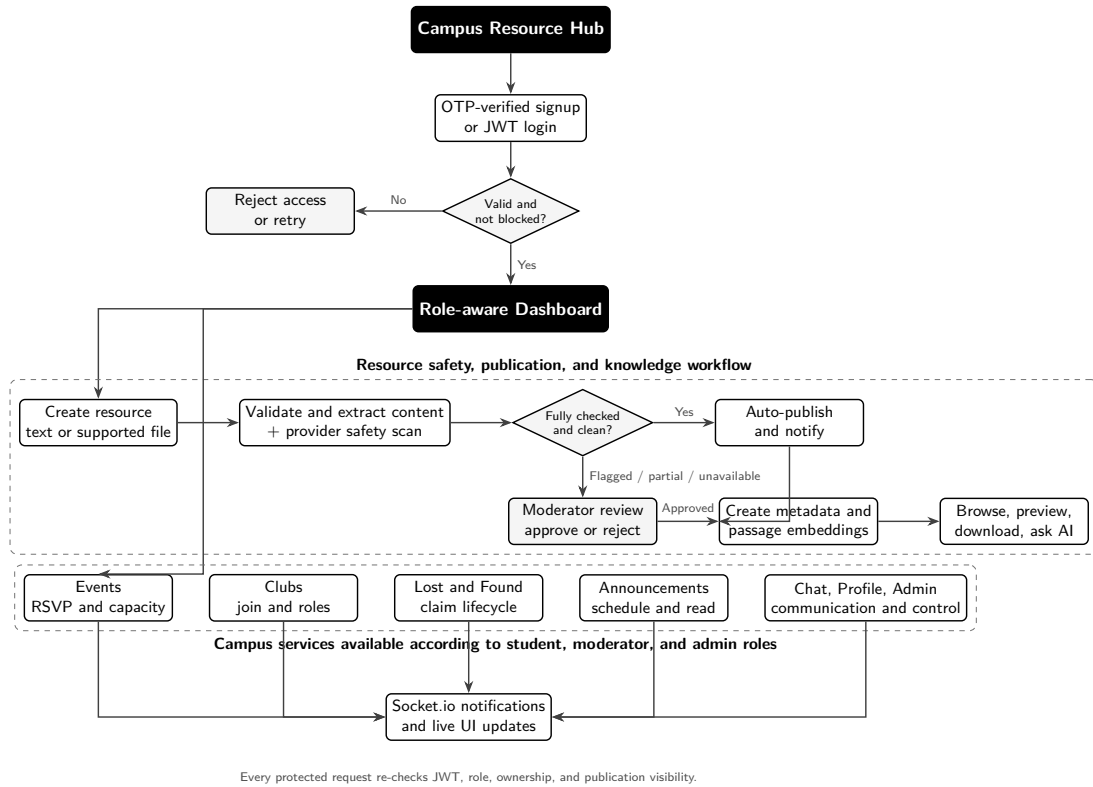


Figure 4.2: User access, resource moderation, publication, campus services, and notification workflow

Table 4.2: User roles and their permissions

Role	Permissions
Student	Browse/search all published content; upload resources; post lost-and-found items; RSVP to events; join clubs; chat; download resources; manage own profile.
Moderator	All student permissions, plus: review resources and lost-and-found posts; inspect AI-flagged items; create and manage events, clubs, and announcements; block/unblock users.
Admin	All moderator permissions, plus: change user roles; delete users; full platform statistics.

4.4 Database Design

The database consists of twelve MongoDB collections. Their fields and relationships are shown in the entity–relationship diagram (Figure 4.3) and summarized in Table 4.3. The `User` collection is the hub: nearly every other collection references it. Moderation fields are stored on the resource and lost-and-found models, while staff-created modules apply their own validation and publication rules.

Table 4.3: Summary of database collections

Collection	Purpose and key fields
User	Account data: name, unique e-mail and student ID, department, role, bcrypt-hashed password, block status.
AuthOtp	Hashed signup / password-reset OTPs with TTL index for automatic expiry.
Resource	Study material metadata: course, department, semester, file info, Cloudinary IDs, moderation verdict, downloads, rating.
Announcement	Notice content, priority, scheduled <code>publishAt</code> , expiry, read receipts, attachments.
Event	Date/time/location, capacity (0 = unlimited), RSVP registrations (going / maybe / declined), club reference.
Club	Name, category, join policy (open / request), members with roles (member / officer), pending join requests.
LostFoundItem	Item type (lost / found), private contact info, claim list with statuses, resolution timestamps.
Conversation	Direct or group chat thread: members, admins, last message reference.
Message	Sender, text, attachments, reply reference, delivered-to and seen-by lists.
Notification	Per-user feed entries with type, link, and read flag.
Embedding	384-dimensional vector, source type + ID (polymorphic), content hash to skip unnecessary re-embedding.
ResourceChunk	Ordered text or safe-image-description passage, parent resource, passage type, and 384-dimensional embedding.

4.5 API Design

The REST API is organized into nine route groups, protected by two middleware layers: `protect` (JWT verification) and `authorize` (role check).

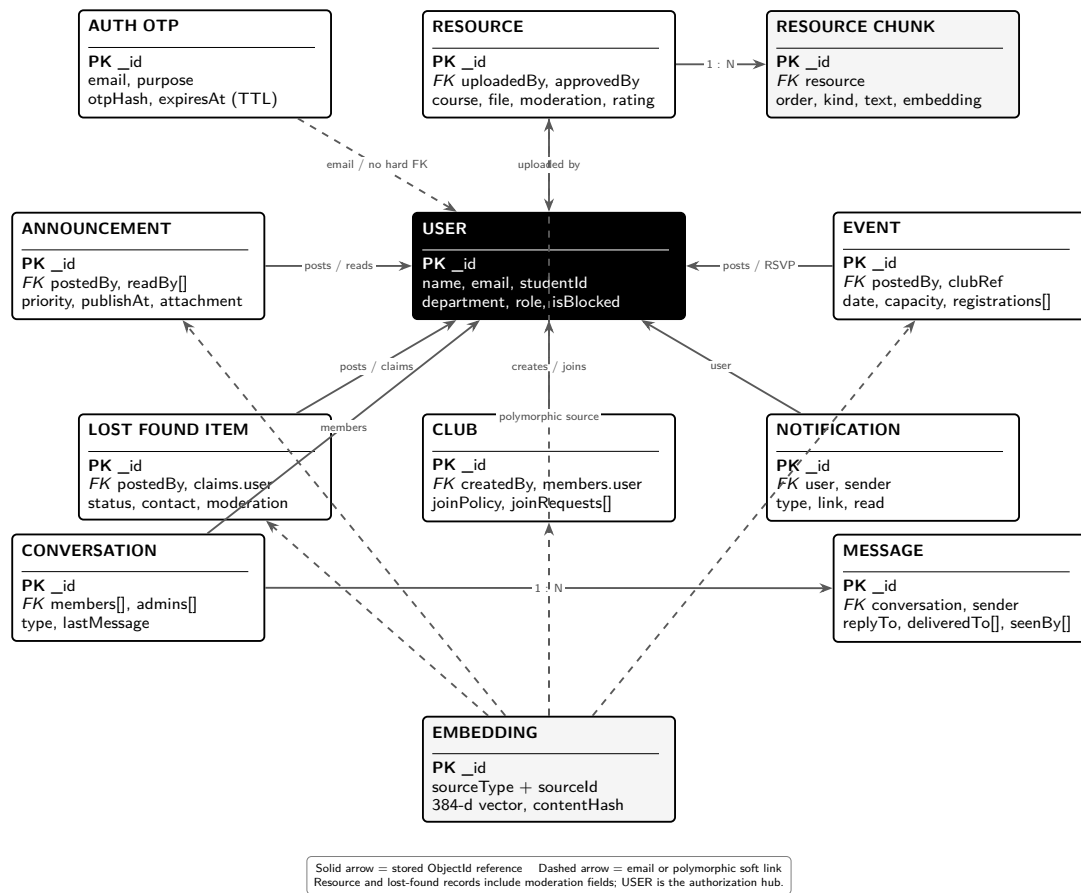


Figure 4.3: MongoDB collections for authorization, academic content, communication, and AI-assisted retrieval

Table 4.4: REST API route groups (representative endpoints)

Group	Base path	Representative operations
Auth	/api/auth	signup, verify-signup-otp, login, forgot/reset password, profile update
Resources	/api/resources	upload, list/filter, preview, file stream, download count
Announcements	/api/announcements	create, approve, pin, mark read, attachment stream
Events	/api/events	create, approve, register, RSVP update, cancel
Clubs	/api/clubs	create, join/leave, decide request, set member role
Lost-Found	/api/lost-found	post, claim, decide claim, resolve, re-open
Chat	/api/chat	conversations, messages, group management, AI assistant (stream)
Notifications	/api/notifications	list, mark read, clear
Admin	/api/admin	statistics, user list, role change, block, delete

4.6 Security Design

Security measures are applied at every layer:

- **Account security:** e-mail ownership proven by OTP before account creation; OTPs stored only as hashes with automatic TTL expiry and attempt counting; passwords hashed with bcrypt (salt rounds = 10).
- **Transport / headers:** HTTPS everywhere, Helmet security headers, strict CORS allow-list.
- **Abuse prevention:** express-rate-limit (3,000 requests per 15 minutes per IP); upload size and type validation with Multer.
- **Authorization:** JWT verification middleware plus role checks on every privileged route; chat access verified per conversation membership.
- **Content safety:** automated screening on resources and other safety-sensitive publishing flows; flagged or partially inspected items remain unpublished for human review.
- **Privacy:** lost-and-found contact details hidden by default and revealed only to approved claimants; passwords excluded from all API responses.

Chapter 5

Methodology

5.1 Implementation Workflow

The system was implemented as a sequence of independently verifiable modules. Data models and protected API routes were completed before their corresponding user interfaces, followed by real-time functions, content processing, semantic indexing, and administrative controls. Automated tests and role-based manual checks were repeated after integration.

After a successful login, every user lands on a role-aware dashboard (Figure 5.1) that summarizes platform activity — total resources, announcements, upcoming events, and open lost-and-found items — and provides quick access to recent resources and the latest announcements. The persistent left navigation and the top “Ask AI” search bar are available from every page.

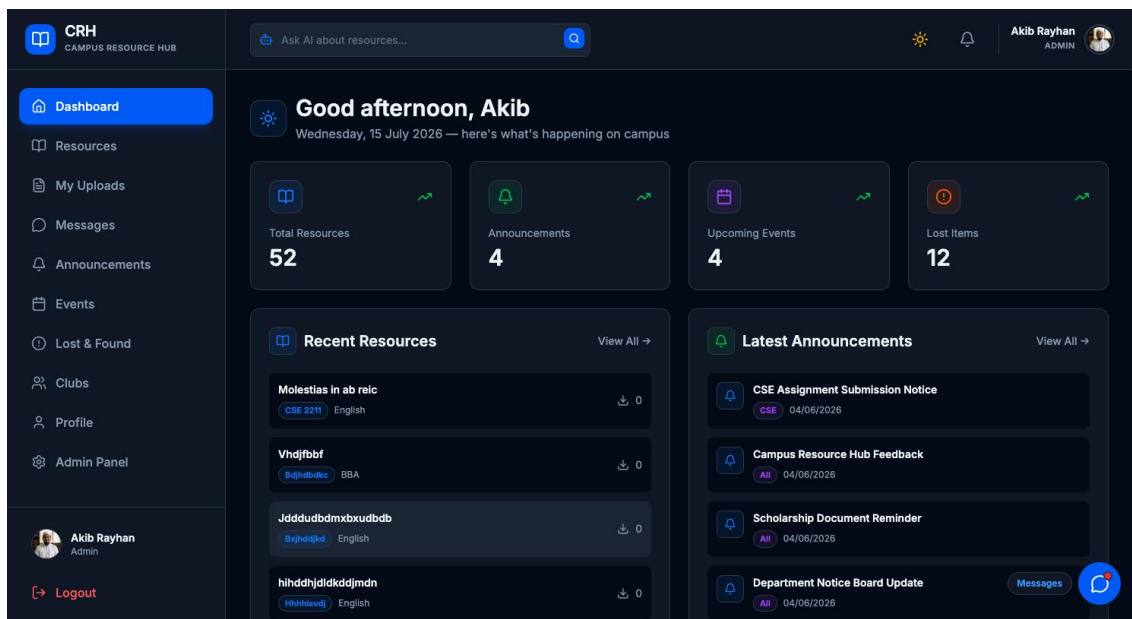


Figure 5.1: Role-aware dashboard with activity summary cards, recent resources, and latest announcements

5.2 Authentication and OTP Verification

Signup collects name, e-mail, student ID, department, and password. The server generates a six-digit OTP, stores its hash in the `AuthOtp` collection with a TTL expiry, and e-mails it via the configured e-mail provider chain. Only after correct OTP entry is the account created with

the default `student` role. Login returns a signed JWT; a parallel OTP flow handles forgotten passwords.

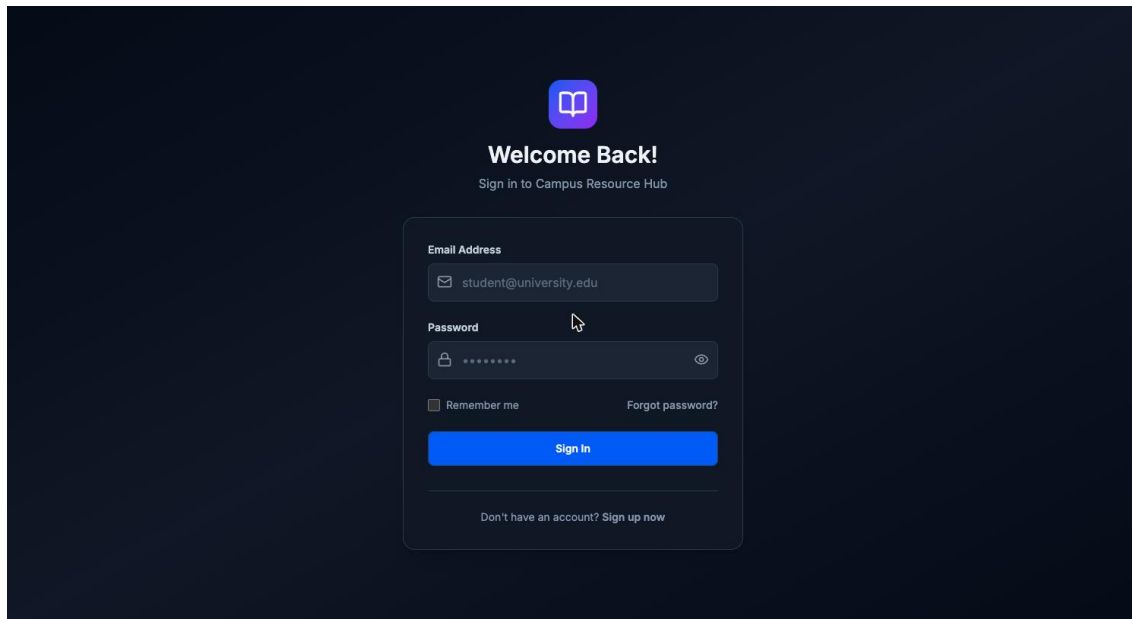


Figure 5.2: Login and signup with e-mail OTP verification

5.3 Resource Upload and AI Moderation Pipeline

Users can author in-application text or blog resources and upload PDF, DOCX, PPTX, XLSX, image, TXT, or ZIP attachments with course, department, and semester metadata. The file goes to Cloudinary; text and images are then extracted server-side (pdf-parse, Mammoth, canvas) and scanned through a provider chain: Groq first, Hugging Face second, and OpenAI last. Clean, fully inspected uploads are published automatically. Flagged files, partial extractions, and provider-unavailable checks fail safely into the moderator review queue; they are never auto-published. Rejection notifies the uploader with a reason. Searchable passages are generated asynchronously for approved text and safe image descriptions.

5.4 Events with Capacity-Aware RSVP

Moderators create events with date, time, location, and optional capacity. Students RSVP as *going*, *maybe*, or *declined*; registration is blocked once capacity is reached. Users see their events in a calendar view, and posters or admins can cancel events, which notifies registrants.

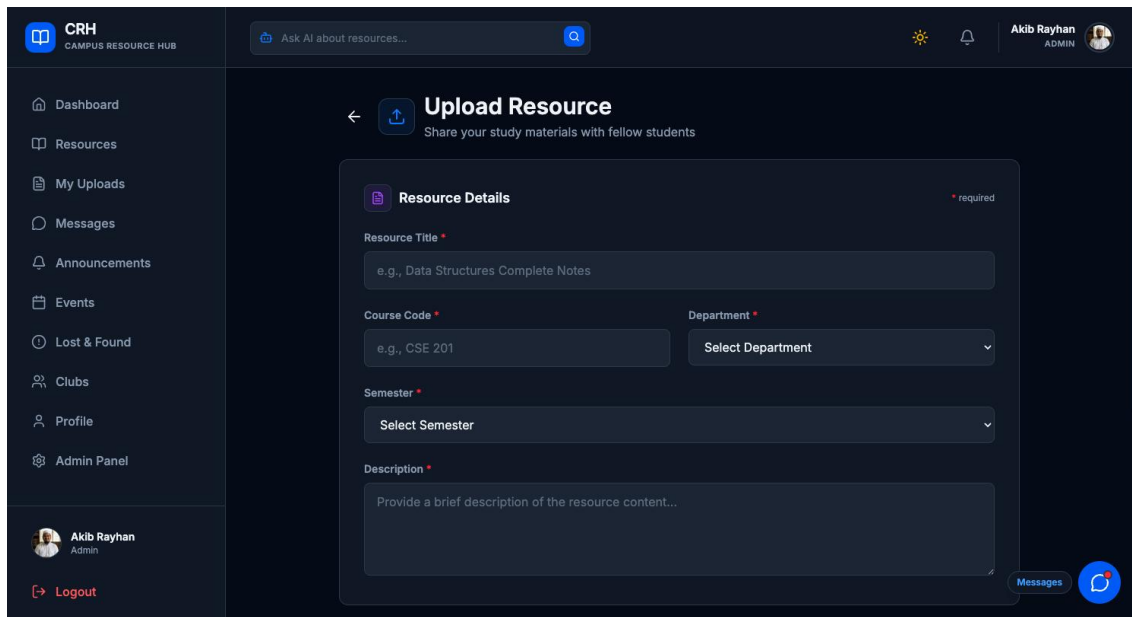


Figure 5.3: Resource upload with metadata and moderation status

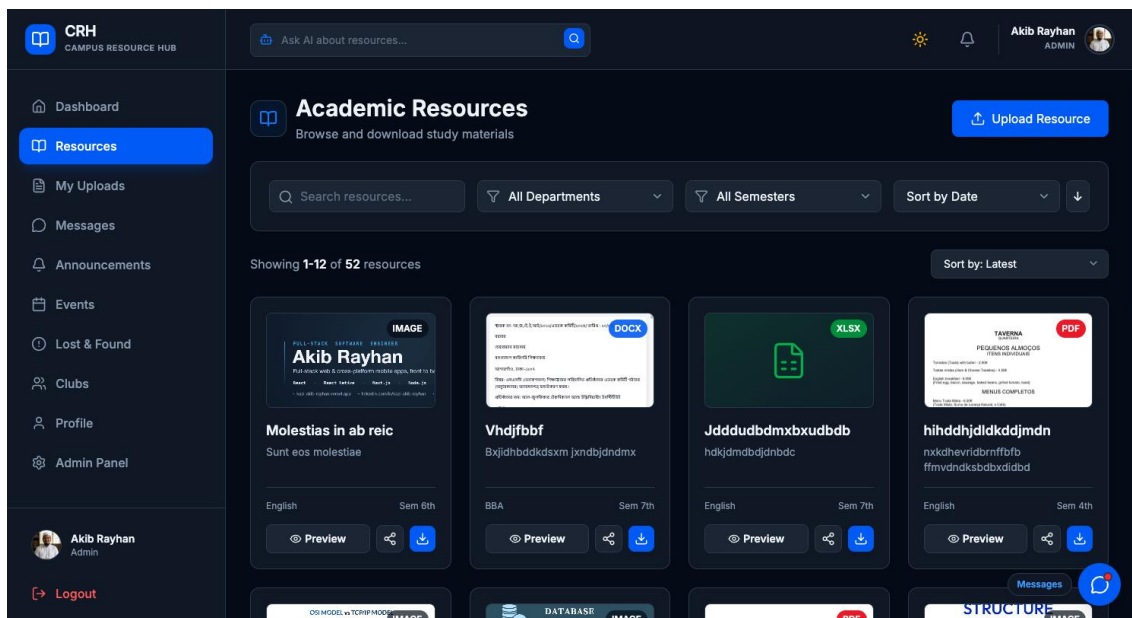


Figure 5.4: Browsing, filtering, previewing and downloading published resources

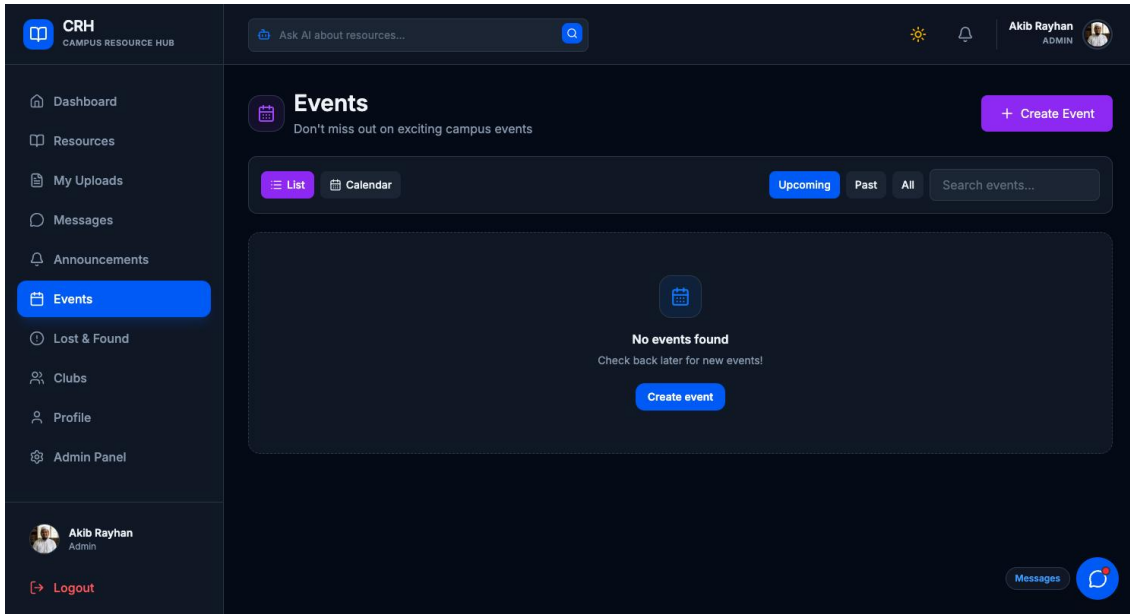


Figure 5.5: Event listing and RSVP flow

5.5 Clubs with Join Policies and Member Roles

Each club declares a join policy: *open* (instant membership) or *request* (application with an optional note, decided by an officer or the creator). Members hold *member* or *officer* roles; officers manage requests and membership.

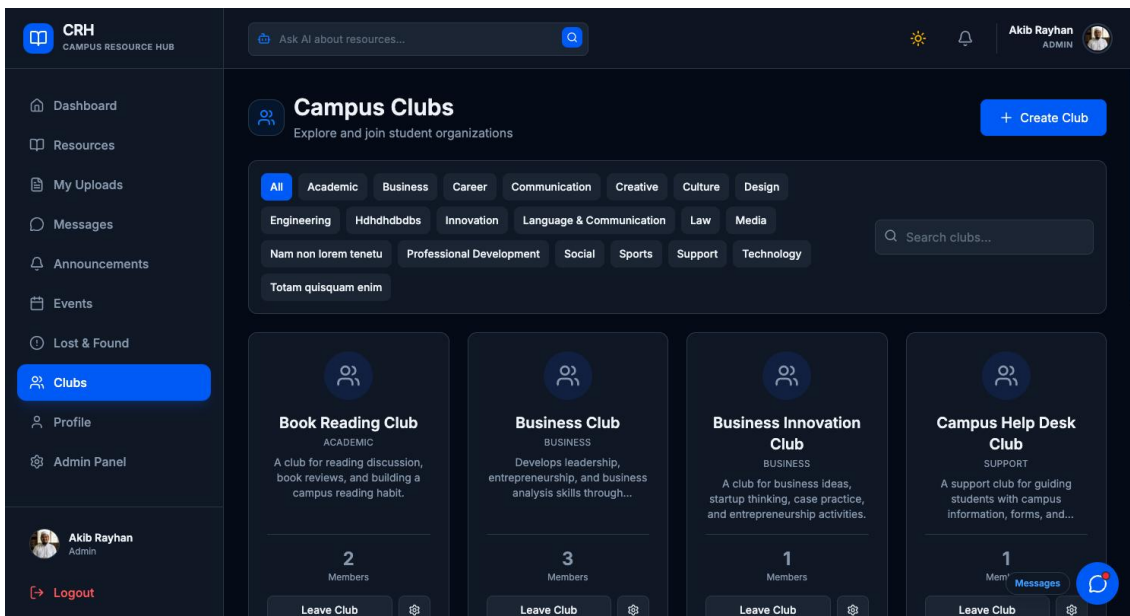


Figure 5.6: Club directory and membership management

5.6 Lost and Found with Claim Workflow

Users post lost or found items with photos. Contact information is hidden by default. Another user files a claim with a note; the poster (or a moderator) approves or rejects it. Approval reveals contact details to the claimant only and moves the item to *claimed*, then *resolved*. This protects personal data while still reuniting items with owners.

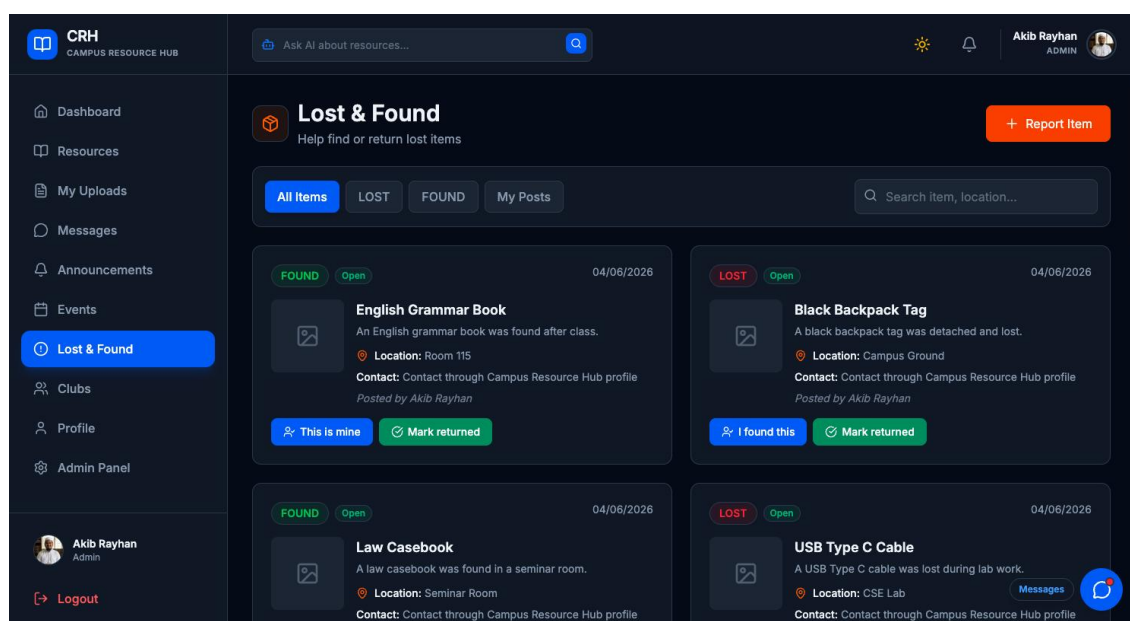


Figure 5.7: Lost-and-found listing and claim dialog

5.7 Announcements with Scheduled Publishing

Announcements carry a priority (normal / important / urgent), optional attachments, a scheduled publishAt time, and an optional expiry. The scheduled publication time controls visibility; persistent background notification scheduling remains deployment-dependent. Read receipts record which students have seen each notice.

5.8 Real-Time Chat

Chat supports direct and group conversations over Socket.io. Messages support attachments, replies, and editing; the system tracks per-user seen status. Group admins can rename groups, add or remove members, and promote other admins. All chat REST endpoints re-verify conversation membership server-side.

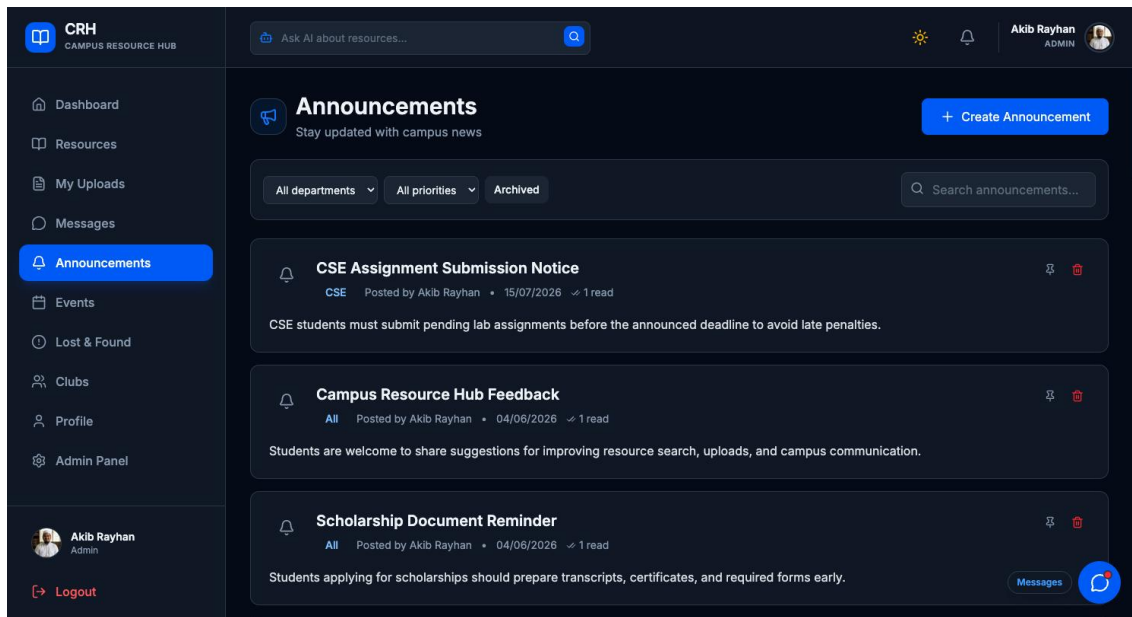


Figure 5.8: Announcement feed with priority badges

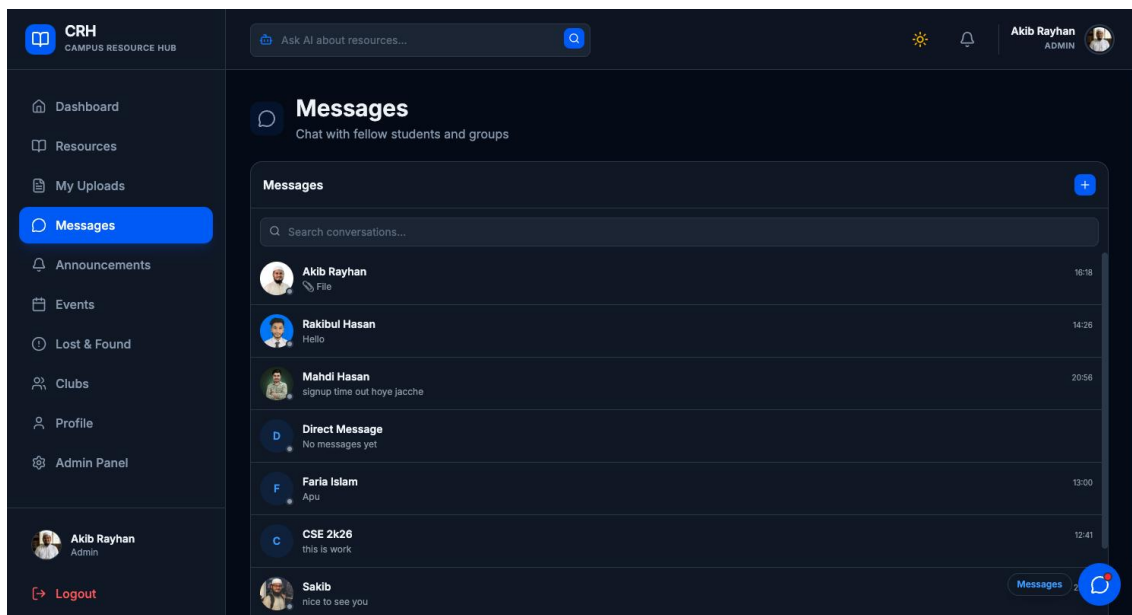


Figure 5.9: Real-time direct and group messaging with seen receipts

5.9 Notifications

Every significant action — new message, approved or rejected upload, event change, claim decision, announcement — creates a notification document and pushes it live over Socket.io. The bell menu supports mark-as-read, mark-all, and clear.

5.10 Semantic Search and AI Assistant

When content is published, its title and body are embedded with a multilingual MiniLM sentence-transformer running inside the Node process (Transformers.js); the 384-dimensional unit vector is stored in the `Embedding` collection, one shared collection for all five content types. A content hash prevents redundant re-embedding when text has not changed. Queries embed the user’s question and run Atlas `$vectorSearch`; results are re-checked against each source collection’s visibility rules, so search can never leak unpublished content. The AI assistant wraps this pipeline and streams its answer token-by-token over server-sent events. The assistant is reachable from the “Ask AI about resources” bar at the top of every page (visible in Figure 5.1).

For resource files, extracted text and safe image descriptions are split into overlapping ordered chunks and embedded separately in the `ResourceChunk` collection. Retrieval first applies the current user’s resource-visibility filter, then ranks matching passages and metadata together. The assistant can therefore answer questions about what a PDF contains, summarize a DOCX file, or describe an indexed image while citing numbered, clickable source cards. Exact-title and lexical-passage matches outrank weaker legacy semantic matches.

5.11 Admin Panel

The panel shows platform statistics (users, resources, pending approvals), a user-management table (block/unblock, delete, role change — role change restricted to admins), and review queues for pending resources and lost-and-found items, together with management tables for staff-created content.

5.12 Deployment

The web frontend and the current backend entry points are configured for Vercel deployment. The live web application is available at [Campus Resource Hub](#). MongoDB Atlas stores application documents, embeddings, and vector-search indexes, while Cloudinary stores resource

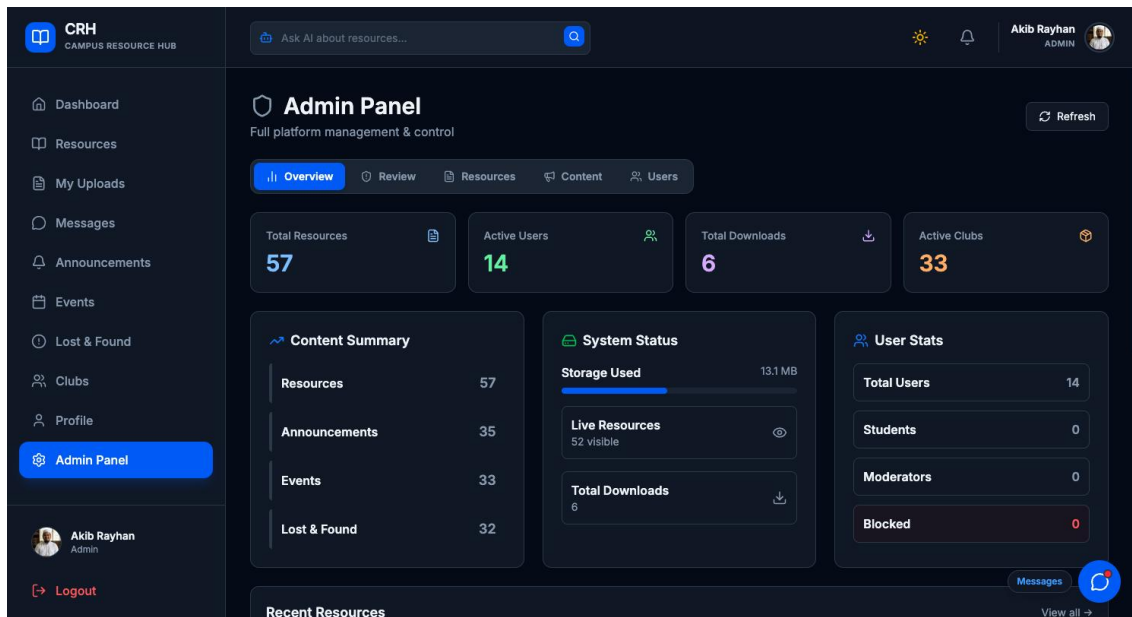


Figure 5.10: Admin panel: statistics and review tools

files and images. Environment variables isolate provider credentials and deployment-specific settings. The companion React Native application currently presents the deployed web application inside a WebView with back-navigation and retry handling; a fully native API client remains future work.

Chapter 6

Results and Analysis

6.1 Achieved Features

All objectives set in Chapter 1 were implemented and verified manually across the three roles:

Table 6.1: Feature completion summary

Feature	Status / observed behaviour
OTP signup & JWT login	Accounts created only after correct OTP; expired OTPs auto-deleted by TTL index.
Resource sharing	Upload, preview, download, and view counting work for supported file resources; in-application text resources are also supported.
AI moderation	Harmful, partially inspected, or provider-unavailable resource uploads are quarantined; clean fully inspected files may auto-publish.
Review workflow	Available for resources and lost-and-found; rejection reasons are delivered as notifications, while staff-created events and announcements follow their own validation rules.
Events	Capacity limit enforced at registration time; RSVP status editable; calendar view correct.
Clubs	Open clubs join instantly; request clubs require officer decision; roles enforced.
Lost & found	Contact hidden until claim approval; item lifecycle open → claimed → resolved.
Chat	Real-time delivery with seen receipts; group administration functions.
Semantic search	Meaning-based matches returned with sources; AI assistant streams answers.
Admin panel	Statistics, user management and resource and lost-and-found review queues operational.

6.2 Security Analysis

The two-layer moderation pipeline proved effective in testing: AI-flagged uploads never became publicly visible, while clean and fully inspected uploads were auto-published. Partial

extraction or provider failure sent content to human review instead of failing open. Rate limiting blocked request floods with HTTP 429 responses. JWT tampering attempts were rejected by signature verification, and role-guarded endpoints returned HTTP 403 for insufficient roles. Lost-and-found contact details were confirmed to be absent from API responses until a claim was approved.

6.3 Bug Report and Resolution Log

The Git history of both repositories was reviewed to distinguish implemented fixes from planned work. Table 6.2 records the principal defects that affected safety, search correctness, real-time connectivity, or usability. Commit hashes provide traceable implementation evidence; merged pull requests represent the authoritative state on the `main` branches.

Table 6.2: Resolved defect log derived from Git history and regression tests

ID	Observed defect and impact	Root cause and implemented resolution	Evidence / status
BR-01	Graphic, adult, or otherwise unsafe files could be approved when a vision provider failed or only part of a document was extracted. This was the highest-severity publication risk.	The earlier path treated incomplete inspection too optimistically. Moderation now fails safe: flagged, partial, and provider-unavailable results remain unpublished for human review. Deterministic text screening and the Groq–Hugging Face–OpenAI provider order were also added.	df5fa96; four moderation-policy regression tests pass. Resolved.

ID	Observed defect and impact	Root cause and implemented resolution	Evidence / status
BR-02	The AI assistant could find a resource by title but could not reliably explain what was inside its PDF or Office document.	Only short metadata and a limited excerpt were searchable. The first repair stored file excerpts (d356801); the completed repair adds ordered overlapping ResourceChunk passages, embeddings, safe image descriptions, and an existing-resource backfill.	d356801, 0e707b3; 54 approved resources indexed with zero extraction failures. Resolved.
BR-03	Content questions could rank weakly related legacy semantic matches above an exact title or file-passage match, producing noisy sources.	Metadata and passage relevance had comparable scores. The final hybrid rank gives exact titles and lexical file passages stronger weight, loads ordered passages for summary intent, and preserves inline source numbering.	0e707b3; real PDF-summary and deadlock-query checks place the relevant Operating Systems resources first. Resolved.
BR-04	A semantic/vector result risked bypassing the visibility rules used by ordinary keyword queries.	Vector candidates were not sufficient authorization evidence. Every candidate is now joined back through the same approved/owner/role filter before its passages are exposed.	c755663, 0e707b3; streaming test returned zero unauthorized sources. Resolved.
BR-05	Real-time chat connections were unreliable after frontend deployment on Vercel.	The Socket.io client transport did not match the deployed network path. The client now prefers explicit WebSocket transport with deployment-compatible connection settings.	42ef620; fix merged to frontend main. Resolved.

ID	Observed defect and impact	Root cause and implemented resolution	Evidence / status
BR-06	Expanding the header AI input to multiple lines changed the header height and obscured adjacent controls; an interim revert also removed desired behavior.	The textarea participated in normal header layout. It now grows in an absolutely positioned layer, retains Enter/Shift+Enter behavior, and shows readable Markdown plus numbered source cards.	757141d, f4259f7, 16edb5f, 641b1bc; production build passes. Resolved.

6.3.1 Current Verification Snapshot

The current repository suites were executed after the latest pull. The backend completed 27 of 27 tests, the frontend completed 6 of 6 tests, and the mobile wrapper completed 1 of 1 test. These suites cover security-sensitive moderation behavior, search authorization, client behavior, and mobile rendering at the unit or integration level. They do not replace a controlled production load test, but they provide repeatable evidence that the checked application paths pass in the current source state.

6.3.2 Open Operational Constraint

Third-party artificial-intelligence services impose quotas and can become temporarily unavailable. The application handles this condition conservatively: content that cannot be inspected completely is not auto-published, and configured fallbacks are attempted where appropriate. Provider availability and credit usage must therefore be monitored independently from functional application defects.

6.4 Verification and Capacity Limitations

No controlled concurrency or endurance benchmark is stored in the repositories, so earlier capacity figures are not presented as measured results. Table 6.3 reports the reproducible automated verification that is available. Before campus-wide deployment, the system should be tested with representative file uploads, semantic searches, simultaneous Socket.IO connections, and third-party provider failures under monitored production-like conditions.

Table 6.3: Automated test-suite results for the current repositories

Application	Passed	Failed	Result
Backend API and services	27	0	Pass
Frontend application	6	0	Pass
Mobile wrapper	1	0	Pass

6.5 Cost and Deployment Considerations

The development deployment uses managed services including Vercel, MongoDB Atlas, Cloudinary, and external e-mail and artificial-intelligence providers. Free service tiers can support development and demonstration, but their quotas, cold starts, execution limits, and provider availability are operational constraints rather than guaranteed capacity. Production adoption therefore requires measured load testing, quota monitoring, a background-job strategy for content processing, and a documented service budget.

Conclusion and Future Work

This project set out to replace the fragmented, unmoderated channels through which campus information currently flows with a single, secure, intelligent platform. The delivered system — Campus Resource Hub — implements verified onboarding, a categorized study-resource repository, capacity-aware events, club management, a privacy-preserving lost-and-found process, scheduled announcements, real-time messaging with seen state, and a semantic AI assistant, supported by targeted content-safety workflows and role-based access control. The implementation demonstrates that modern techniques usually associated with large commercial platforms — AI content moderation, vector-based semantic search, streamed LLM assistance — can be integrated into a student project integrated with managed cloud services. All current automated suites passed, while production-scale concurrency remains to be established through controlled load testing. Beyond its immediate utility, the project provided deep practical experience in full-stack architecture, real-time systems, applied AI services, and cloud deployment.

Future Work

Planned improvements include:

1. **Fully native mobile client** — replace the implemented React Native WebView shell with dedicated native screens that consume the existing REST and real-time interfaces.
2. **Background job queue** — moving AI scanning and embedding generation off the request path (e.g. BullMQ) to remove the largest CPU spike.
3. **Push notifications** — Web Push / FCM so users are reachable when the site is closed.
4. **Richer analytics** — per-department dashboards, trending resources, and event attendance reports.
5. **Federated login** — university e-mail SSO (OAuth2) alongside OTP signup.
6. **Horizontal scaling** — multiple backend instances with a Socket.io Redis adapter once campus-wide load requires it.
7. **Accessibility and localization** — full Bangla interface and WCAG-compliant UI review.

References

- [1] Meta, “Facebook groups.” <https://www.facebook.com/help/162866443847527>. Accessed: October 2026.
- [2] WhatsApp, “Whatsapp features.” <https://www.whatsapp.com/features>. Accessed: October 2026.
- [3] Moodle, “Moodle learning platform.” <https://moodle.org/>. Accessed: October 2026.
- [4] Google, “Google classroom.” <https://edu.google.com/workspace-for-education/products/classroom/>. Accessed: October 2026.
- [5] Google, “Google drive.” <https://www.google.com/drive/>. Accessed: October 2026.
- [6] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 3982–3992, 2019.
- [7] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 5776–5788, 2020.
- [8] MongoDB, Inc., “Mongodb atlas vector search documentation.” <https://www.mongodb.com/docs/atlas/atlas-vector-search/>. Accessed: October 2026.
- [9] Meta Platforms, “React documentation.” <https://react.dev/>. Accessed: October 2026.
- [10] OpenJS Foundation, “Express web framework documentation.” <https://expressjs.com/>. Accessed: October 2026.
- [11] MongoDB, Inc., “Mongodb documentation.” <https://www.mongodb.com/docs/>. Accessed: October 2026.
- [12] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” Tech. Rep. RFC 7519, Internet Engineering Task Force, 2015.
- [13] Socket.IO, “Socket.io documentation.” <https://socket.io/docs/>. Accessed: October 2026.
- [14] Cloudinary, “Cloudinary documentation.” <https://cloudinary.com/documentation>. Accessed: October 2026.

Appendix A

Supplementary Material

A.1 Code Availability

The project source is maintained in the following repositories:

- **Live web application:** [Campus Resource Hub](#)
- **Web frontend:** [campus-resource-hub-frontend](#)
- **Backend API:** [campus-resource-hub-backend](#)
- **Mobile wrapper:** [campus-resource-hub-mobile](#)

A.2 Current Automated Verification

At the time of final formatting, the checked repository state completed 27 backend tests, 6 frontend tests, and 1 mobile test without failures. Production-scale load testing and formal user-acceptance testing remain outside the scope of these automated suites.